

ТЗ: игра «Путь» — MVP

Oct 3, 2026 · @Pavel

Цель и концепция

«Путь» — браузерная карточная игра (PWA, mobile-first) о внутреннем росте взрослого человека. MVP: 2 главы, 53 карточки с выбором, прохождение за 15–25 минут.

- **Жанр:** нарративная карточная игра со свайпами, референс — Reigns.
- **Суть:** герой живёт во вневременном городке. Каждый день — серия бытовых ситуаций. Выбор тратит видимые ресурсы и незаметно меняет скрытые качества героя.
- **Рост** виден не в цифрах, а в том, как меняется мир: появляются новые карточки, персонажи раскрываются, приходят озарения.
- **Не религиозная игра:** без религиозной лексики, без морализаторства. Игра никогда не говорит, что выбор был плохим.
- **Цель MVP:** проверить, что игрок чувствует последствия своих выборов и хочет пройти дальше второй главы. Как это измеряется — в разделе «Критерии приёмки».

Аудитория, сеттинг и тон

Игра для взрослых 25–55 лет, которые играют с телефона короткими сессиями по 3–5 минут.

Сеттинг. Вневременной городок у реки: рынок, мастерская героя, мост, дорога из города, дом на холме. Без технологий, без указания эпохи и страны. Герой — гончар средних лет, у него заказы, долги и постоянная спешка.

Постоянные персонажи:

Персонаж	Кто он	Тема
Марта	соседка-вдова	одинокость, внимание
Тимон	торговец, конкурент	честность, зависть
Ян	ученик в мастерской	терпение, наставничество
Илья	старый друг, должен денег	обида, отпускание

Персонаж	Кто он	Тема
Лия	сестра, живёт далеко, пишет письма	близкие, время
Странник	без имени, появляется редко	сострадание, мудрость

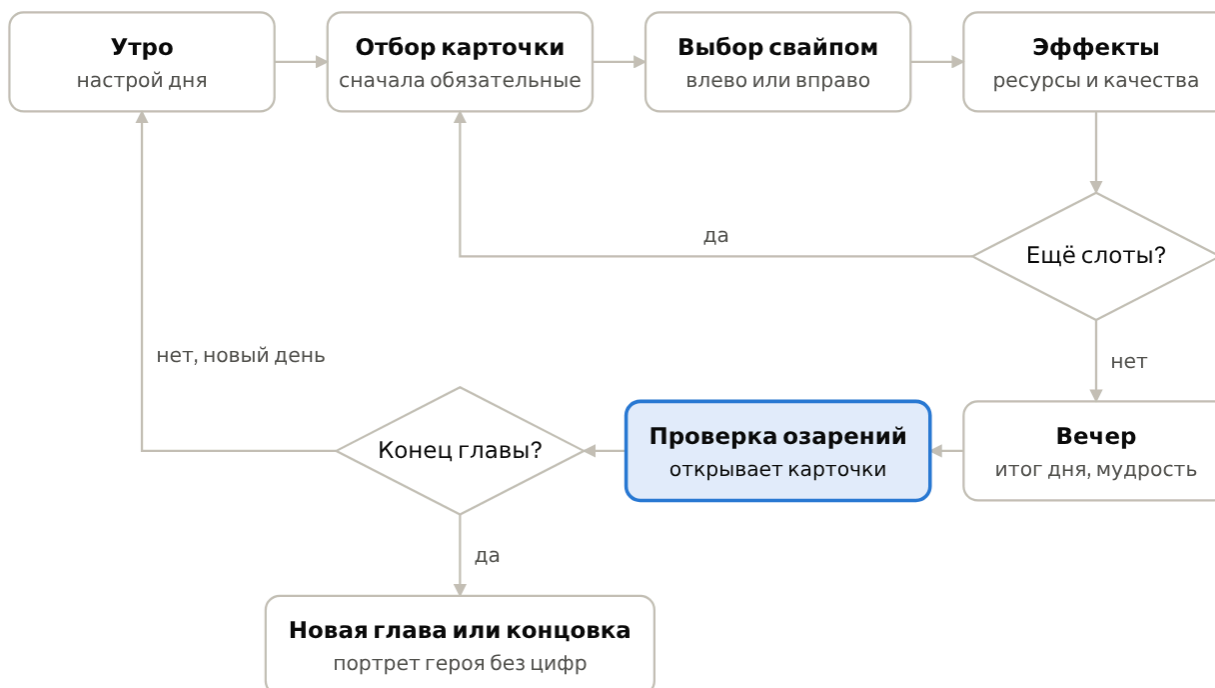
Тон:

- Текст карточки — до 200 символов, спокойно, простыми словами.
- Никаких оценок «правильно / неправильно». Последствия приходят через мир и людей.
- Стоп-лист слов (проверяется скриптом): бог, грех, святой, молитва, церковь, храм, рай, ад — список дополняется.
- Визуал: минимализм, тёплая палитра, одна плоская иллюстрация на персонажа.

Игровой цикл

Одна сессия = один день: утро, 4 ситуации с выбором, вечер, всего 1,5–2,5 минуты. Глава = 5 дней, MVP = 2 главы.

Один день — 4 выбора, рост приходит через озарения



игровой цикл · один день

Озарения проверяются каждый вечер, показываются на экране вечера (слот не занимают) и меняют набор карточек следующих дней — именно так игрок видит рост. Проигрыша нет: ресурс на 0 добавляет кризисную карточку на шаге отбора.

Ресурсы и скрытые качества

Игрок видит 4 ресурса и не видит 5 качеств. Качества конфликтуют между собой и с ресурсами, поэтому одного оптимума «растить всё» нет.

Видимые ресурсы (0–100, старт 50, изменение за выбор от –20 до +20):

Ресурс	Что значит	Кризис
Достаток	деньги, еда, товар	«Нужда» при 0, «Избыток» при 100
Силы	телесная энергия	«Изнеможение» при 0
Покой	внутреннее состояние	«Тревога» при 0
Близкие	тепло отношений	«Одиночество» при 0

«Избыток» — кризис сверху: деньги есть, а времени на людей и себя нет; давит на Близких и Покой. Так жадная игра на ресурсы тоже упирается в предел.

Скрытые качества (–20...+20, старт 0):

Качество	Растёт, когда герой...
Внимательность	замечает людей и детали, останавливается
Честность	говорит правду, в том числе себе
Сострадание	помогает ценой своих ресурсов
Отпускание	перестаёт держаться за обиду, контроль, статус
Смелость	действует, когда страшно или невыгодно

Изменения качеств:

- Обычный выбор — ± 1 . ± 2 — только в ключевых сюжетах. Больше 2 по модулю запрещает валидатор.
- Вариант меняет не больше 2 качеств.

- Не меньше трети situation и chain — конфликтные по формальному определению ниже.

Пары конфликтов качеств:

Конфликт	Пример ситуации
Честность ↔ сострадание	сказать Яну, что его работа плоха, или пощадить
Смелость ↔ отпускание	бороться с Тимоном за место на рынке или уступить
Честность ↔ отпускание	докопаться до правды о долге Ильи или отпустить
Внимательность ↔ смелость	присмотреться ещё или действовать сейчас

Карточка **конфликтная**, если для одной из пар $X \leftrightarrow Y$: варианты растят разные качества пары (А: $X+$, Б: $Y+$), либо один вариант растит X и снижает Y . Цена в ресурсах конфликтом не считается: её имеет почти любая карточка.

Порядок кризиса (всё внутри `applyChoice`):

1. Применяются эффекты выбора, ресурсы ограничиваются 0–100.
2. Если это была кризисная карточка: её ресурс становится `max(значение, 40)`, у «Избытка» — `min(значение, 60)`, карточка уходит из очереди.
3. Из очереди `pendingCrises` удаляются записи, чей ресурс уже не на краю.
4. Каждый ресурс на 0 (Достаток — и на 100) ставит свой кризис в конец очереди, если его там нет.
5. Кризисы идут по одному в следующие слоты (если день кончился — с утра). Два ресурса на 0 — два кризиса в двух слотах. При показе край проверяется снова.
6. Кризис повторяемый (`once: false`), повтор показывает второй вариант текста (`textVariants` по `shown`). Это не проигрыш. Цель — не больше 4 кризисов за прогон («Критерии приёмки»).

Отображение. Качества цифрами не показываются (кроме debug-режима). Перед свайпом над ресурсами, которые изменятся, появляются точки: размер = сила изменения, знак скрыт (как в Reims).

Карточки: типы и логика выбора

Карточка — ситуация и два варианта. Все карточки в JSON, код от контента не зависит. Утро, вечер и озарения — не карточки, а экраны дня, и в пул отбора не попадают.

Тип	Назначение	Как попадает в слот	Повтор
situation	ситуация главы	случайно по весу	нет
routine	бытовая ситуация, <code>chapter: 'any'</code>	случайно по весу	да, <code>cooldownDays: 3</code>
chain	продолжение ранее сделанного выбора	по расписанию	нет
crisis	ресурс на 0 или Достаток на 100	из очереди кризисов	да

Отдельного типа «исправление» нет: возврат к поступку — обычная chain-ситуация (раздел «Озарения, мудрости, концовки»).

Routine-карточки закрывают слоты, когда situation и chain кончились: 10 штук на обе главы. Через `textVariants` они откликаются на качества героя, так мир реагирует на героя уже в главе 1.

Отбор карточки в слот (в порядке приоритета):

1. Первый кризис в очереди `pendingCrises`, чей ресурс всё ещё на краю.
2. Карточка с наступившим `mustShowBy` (день настал, а она ещё не показана), самая ранняя первой. Так гарантируются ключевые сюжеты: до этого дня они выпадают случайно, в этот день — обязательно.
3. Отложенная chain с `day <= сегодня` и выполненным `requires`, самая ранняя первой (пропущенные не теряются).
4. Взвешенная случайная situation или routine из допустимых: текущая глава или `any`, `requires` выполнено, `once` не исчерпан, `cooldownDays` прошёл. Situation текущей главы весит x3 к routine.
5. Допустимых нет — слот пустой, день кончается раньше. Для MVP это ошибка контента: `simulate.ts` требует 0 пустых слотов.

Отбор — чистая функция. `drawCard(state)` ничего не меняет: не достаёт кризис из очереди и не снимает отложенные, а только пропускает неподходящие. Все мутации — в `applyChoice` и переходах дня: там кризис

уходит из очереди, а вечером снимаются отложенные, чей `requires` уже невыполним (с записью в лог). Поэтому `draw(state)` идемпотентна, в том числе после `save/load`, и это покрыто тестом.

Стороны свайпа. Варианты хранятся как `choices[0..1]` без привязки к стороне. Сторона выбирается случайно при показе, чтобы игрок не выучил «дорогое действие всегда справа». Исключение — флаг `fixedSides` для редких карточек, где порядок важен.

Случайность без состояния: `rng = mulberry32(hash(seed, day, slot, salt))`. Одинаковые `seed` и выборы дают одинаковую игру, а карточку и стороны любого слота можно пересчитать при загрузке.

Варианты текста (`textVariants`, берётся первый подходящий): та же ситуация описана иначе после озарения, флага, повторного показа или при ведущем качестве.

Озарения, мудрости, концовки

Рост героя игрок видит через четыре системы, и ни одна из них не оценивает выбор.

Правило следа. Каждый вариант `situation` и `chain` оставляет след, который что-то меняет дальше: отложенную карточку, мудрость или флаг, у которого есть читатель (`requires`, `textVariants` или условие концовки). Флаг, который никто не читает, следом не считается. `Routine` и `crisis` от правила освобождены. Правило проверяет `check-content` (мёртвые флаги — ошибка).

Мудрости. Фраза или мини-притча до 280 символов, без источника. Показывается на экране вечера и попадает в «Записи». Мудрость бывает у любого варианта и не склоняет ни к одному из них. **Тест перестановки** (ручная вычитка контента): подставьте мудрость к другому варианту — она должна звучать естественно. Примеры к сюжету «Раненый на дороге»:

- остановился — «У каждой дороги две цены: одну платишь сразу, другую — потом.»
- проехал мимо — «Торг не ждёт. Дорога тоже.»

Озарения (в MVP — 2). Проверяются только вечером, начиная с 6-го дня. Если условие выполнено, ставится `pendingInsight`, и озарение показывается на экране вечера после мудрости. Слот не занимает, выбора нет, у озарения один набор `effects`. Не больше одного за вечер.

Озарение	Условие (старт)	Что меняется
«Тишина»	внимательность ≥ 8	3 новые карточки, где герой слышит невысказанное; у 3 карточек меняется текст
«Взгляд»	сострадание ≥ 8	Странник и Марта получают свои истории (3 новые карточки)

Возвращение к поступкам. Поступок ставит флаг и планирует chain через фиксированное `inDays` (своё у каждой карточки). Продолжение подаётся как обычная ситуация: Тимон возвращается к разговору, Илья встречается на мосту. Игроку не говорят, что он ошибся, и не предлагают «исправиться».

Концовки (2 «среза» в конце главы 2). Каждая — портрет героя словами, без цифр, с фразами про 2 ведущих качества. Обе — полноценные финалы, не награда и не штраф, поэтому порог калибруется на разброс, а не на редкость.

Концовка	Условие (старт)	Смысл
«Колесо»	иначе	жизнь идёт своим ходом; герой многое успевает и иногда чувствует, что что-то проходит мимо
«Первый шаг»	сумма качеств ≥ 20 и хотя бы 1 озарение	герой впервые видит свою жизнь со стороны и делает первый шаг

Ведущее качество и ничья. Ведущие качества сортируются по значению; при равенстве выше то, что росло позже (по `history`), дальше — порядок из типа `Quality`. Одно правило для `topQuality`, утренних текстов и портрета в концовке.

Все пороги — стартовые. Финальные значения берутся из `simulate.ts` до написания основного контента. Главная цель: у человекоподобной политики «Первый шаг» в 30–70% прогонов; остальные цели — в «Критериях приёмки».

Контент MVP: главы 1–2

MVP — 10 игровых дней по 4 слота = 40 слотов. Контент закрывает их с запасом даже без озарений: 53 карточки с выбором (106 вариантов), 16 мудростей, 25 текстов дня.

- **Глава 1 «Шум» (дни 1–5).** Заказы, долги, спешка. Герой почти не замечает людей. Задача главы — показать цену суеты.
- **Глава 2 «Пробуждение» (дни 6–10).** Мир замедляется, у героя появляются вопросы к себе, становятся доступны озарения. Переход — после вечера 5-го дня.

Тип	Глава 1	Глава 2	Всего
situation	12	10	22
situation, открываются озарениями	—	6	6
routine (any , повторяемые)	—	—	10
chain	5	5	10
crisis (второй текст на повтор)	—	—	5
озарения (экраны)	—	2	2
мудрости	8	8	16
тексты утра / вечера (по одному на день)	5 + 5	5 + 5	20
варианты утра по ведущему качеству	—	—	5

Покрытие слотов. В каждой главе 20 слотов закрывают situation главы, до 5 chain и пул из 10 routine (каждая — до 2 раз за главу при `cooldownDays: 3`). Игрок без озарений пустых дней не получает. Итог проверяет `simulate.ts`.

Тексты дня. Базовые тексты утра и вечера привязаны к дню (`DayText.day`). Варианты по ведущему качеству — только утренние, по одному на качество. Утро берёт вариант вместо базового текста, когда ведущее качество (≥ 3) сменилось с прошлого показа варианта, и не чаще раза в 3 дня. Так рост заметен уже в главе 1, а один текст не повторяется подряд.

Три ключевых сюжета-притчи. Игрок сначала проживает ситуацию, смысл приходит позже; след есть у обоих вариантов. Все три показываются в каждом прохождении за счёт `mustShowBy`.

Сюжет	Показ	Ситуация	Вариант А	Вариант Б
«Раненый на дороге»	гл. 1, mustShowBy: 3	герой спешит на рынок, у дороги лежит Странник	остановился: Достаток −, сострадание +2; через 3 дня Странник приходит в мастерскую	проехал: Достаток +, сострадание −1; через 3 дня встреча со Странником на мосту
«Лишняя сдача»	гл. 1, mustShowBy: 4	Тимон ошибся при расчёте в пользу героя	сказал: Достаток −, честность +2, флаг <code>timon_trust</code> — в главе 2 Тимон предлагает общий заказ (<code>textVariants</code> и отдельный вариант выбора)	промолчал: Достаток +, честность −1; через 2 дня Тимон возвращается к разговору
«Мешок камней»	гл. 2, mustShowBy: 8	Илья пришёл мириться, долг не вернул	простил: отпускание +2, Достаток −; через 2 дня Покой +	не простил: Покой −; через 2 дня «Илья уезжает»

Правило баланса: оба варианта что-то стоят и оставляют след. Отложенные карточки планируются так, чтобы последний возможный день карточки + `inDays` ≤ 10 (проверяет валидатор).

Модель данных

Контент описывается в JSON декларативно: условия — это данные, а не функции. Типы ниже — контракт между контентом и движком, по ним строится Zod-схема для валидации.

```
type Resource = 'wealth' | 'strength' | 'peace' | 'bonds';
type Quality = 'attention' | 'honesty' | 'compassion' | 'letgo' | 'courage';
// порядок = последний tie-break

type Condition =
  | { all: Condition[] } | { any: Condition[] } | { not: Condition }
  | { quality: Quality; gte?: number; lte?: number }
```

```

| { resource: Resource; gte?: number; lte?: number }
| { flag: string }
| { chapter: number }
| { dayGte?: number; dayLte?: number }
| { shown: string } // карточка была показана
| { chose: { card: string; choice: string } } // выбран вариант
| { topQuality: Quality; gte?: number }; // ведущее качество, ничья
– по правилу из ТЗ

interface Effects {
  resources?: Partial<Record<Resource, number>>; // -20..+20
  qualities?: Partial<Record<Quality, number>>; // -2..+2, не больше 2
  качеств
  setFlags?: string[]; // у каждого флага должен
  быть читатель
  clearFlags?: string[];
  schedule?: { cardId: string; inDays: number }[]; // фиксированный срок
  wisdomId?: string;
}

interface Choice { id: string; label: string; effects: Effects; }

type CardType = 'situation' | 'routine' | 'chain' | 'crisis';

interface Card {
  id: string;
  chapter: number | 'any';
  type: CardType;
  character?: string;
  text: string; // до 200 символов
  textVariants?: { when: Condition; text: string }[];
  choices: [Choice, Choice]; // стороны назначаются при
  показе
  fixedSides?: boolean;
  requires?: Condition;
  mustShowBy?: number; // гарантированный показ не
  позже этого дня
  weight?: number; // по умолчанию: situation
  3, routine 1
  once?: boolean; // по умолчанию true;
  routine и crisis – false
  cooldownDays?: number; // routine – 3
  crisis?: { resource: Resource; edge: 0 | 100 }; // только у crisis
}

interface Insight { id: string; title: string; text: string; requires:

```

```

Condition; effects: Effects; }
interface Wisdom { id: string; text: string; }    // до 280 символов

// базовый текст: part + day; вариант по качеству: part 'morning' + quality,
без day
interface DayText {
  id: string;
  part: 'morning' | 'evening';
  day?: number;                                // 1..10
  quality?: Quality;
  text: string;
}
interface Ending { id: string; title: string; text: string; requires?:
Condition; priority: number; }

type Phase = 'morning' | 'slot' | 'evening' | 'insight' | 'chapter' |
'ending';

interface GameState {
  version: number;                            // схема сохранения
  contentVersion: string;                     // хэш контента при сборке
  seed: number;
  chapter: number;
  day: number;                                // 1..10
  slot: number;                               // 0..3
  phase: Phase;
  current?: { cardId: string; leftChoiceId: string }; // карточка на экране и
её стороны
  resources: Record<Resource, number>;        // 0..100
  qualities: Record<Quality, number>;         // -20..+20
  flags: string[];
  shown: Record<string, number[]>;            // cardId -> дни показа
  scheduled: { cardId: string; day: number }[];
  pendingCrises: string[];                    // id кризисных карточек
  pendingWisdoms: string[];
  pendingInsight?: string;
  lastMorningVariant?: { day: number; quality: Quality };
  journal: { day: number; kind: 'wisdom' | 'insight'; id: string }[];
  history: { day: number; slot: number; cardId: string; choiceId: string }[];
}

// отдельный ключ хранилища, не стирается «Начать заново» и сбросом прогресса
interface PlaytestMetrics {

```

```

reachedEnding: boolean;
endingChoice?: 'continue' | 'finish'; // первое нажатие на экране концовки
firstRunHistory?: GameState['history'];
}

```

Пример карточки (сюжет «Раненый на дороге», след у обоих вариантов):

```

{
  "id": "c1_wounded_road",
  "chapter": 1,
  "type": "situation",
  "character": "wanderer",
  "text": "Ты везёшь горшки на рынок, торг вот-вот начнётся. У обочины лежит человек и не встаёт.",
  "mustShowBy": 3,
  "choices": [
    {
      "id": "pass",
      "label": "Проехать мимо",
      "effects": {
        "resources": { "wealth": 10, "peace": -5 },
        "qualities": { "compassion": -1 },
        "setFlags": ["passed_wanderer"],
        "schedule": [{ "cardId": "c1_wanderer_bridge", "inDays": 3 }],
        "wisdomId": "w_road_longer"
      }
    },
    {
      "id": "stop",
      "label": "Остановиться",
      "effects": {
        "resources": { "wealth": -15, "strength": -5 },
        "qualities": { "compassion": 2, "attention": 1 },
        "setFlags": ["helped_wanderer"],
        "schedule": [{ "cardId": "c1_wanderer_returns", "inDays": 3 }],
        "wisdomId": "w_hurry"
      }
    }
  ]
}

```

Файлы контента: `content/cards.ch1.json`, `content/cards.ch2.json`, `content/cards.common.json` (routine и crisis), `content/wisdoms.json`, `content/insights.json`, `content/endings.json`, `content/characters.json`, `content/days.json` (тексты утра и вечера).

Техническая часть

Статическое PWA без бэкенда: вся логика и сохранения — на устройстве игрока, игра работает офлайн.

Часть	Решение
Сборка	Vite + React 18 + TypeScript (strict)
Состояние UI	Zustand
Свайп и анимации	Framer Motion через LazyMotion + domMax (drag требует domMax, а не domAnimation)
PWA	vite-plugin-pwa (Workbox), кэш всех ресурсов, manifest, иконки
Валидация контента	Zod — только в сборке и тестах, в рантайм-бандл не попадает
Тесты	Vitest
Случайность	без состояния: mulberry32(hash(seed, day, slot, salt))
Хостинг	любой статический или свой nginx, HTTPS обязателен для PWA

Архитектура. Движок — чистые функции без React. Отбор ничего не меняет; все мутации состояния — в applyChoice и переходах дня.

```

src/
  engine/          // без зависимости от React
    conditions.ts  // evaluate(cond, state), topQuality с tie-break
    draw.ts        // drawCard(state, content) -> карточка + стороны, без
мутаций
    apply.ts       // applyChoice: эффекты, очередь кризисов, расписание
    day.ts         // startDay, endDay (снятие невыполнимых chain),
checkInsight, nextChapter
    endings.ts     // pickEnding(state, endings)
    rng.ts         // mulberry32 + hash
    save.ts        // migrate, sanitize по contentVersion, validateImport
content/           // JSON + loader
store/            // Zustand: state, save/load, export/import, метрики
ui/              // экраны и компоненты
debug/           // панель отладки
scripts/
  schema.ts       // Zod-схемы
  check-content.ts // статические проверки контента
  stoplist.ts     // стемы стоп-слов + тесты на ложные срабатывания
  simulate.ts     // прогоны по политикам и метрики

```

check-content.ts проверяет:

- схему (Zod) и ссылки `cardId` / `wisdomId` / `character` ;
- правило следа для `situation` и `chain` (`routine` и `crisis` освобождены);
- мёртвые флаги: у каждого `setFlags` есть читатель (`requires`, `textVariants`, условие концовки или озарения), у каждого читаемого флага есть писатель;
- статическую достижимость: на каждую `chain` есть `schedule`, флаги из её `requires` кто-то ставит, у карточек озарений есть озарение-источник, диапазоны `dayGte` / `dayLte` пересекаются с днями главы;
- сроки: последний возможный день карточки (по главе, `dayLte`, `mustShowBy`) + `inDays` ≤ 10 ; `mustShowBy` только у `situation` и не позже конца главы;
- качества: не больше 2 по модулю и не больше 2 качеств на вариант; доля конфликтных карточек по формальному определению из «Ресурсов и скрытых качеств» не меньше трети;
- длину текстов (200 / 280) и стоп-лист.

Стоп-лист. `\b` в JS не работает с кириллицей, а поиск подстроки даёт ложные срабатывания: «бог» в «богатстве», «рай» в «крае», «ад» в «саде». Проверка идёт по стемам со словоформами и Unicode-границами, с флагом `u`:

```
/(?!\\p{L})бог(?:a|y|om|e|i|ов)?(?:\\p{L})/iu
```

simulate.ts. Пишется до основного контента и прогоняет по 1000 игр на политику. При случайных сторонах «всегда влево / вправо» равно случайному выбору, поэтому таких политик нет. Пороги калибруются по human-like.

Политика	Как выбирает	Зачем
random	случайно	база
always-first / always-second	всегда <code>choices[0]</code> / <code>choices[1]</code>	ловит авторское смещение по порядку вариантов
always-costly	вариант с наибольшей потерей видимых ресурсов	эксплойт «дорогое = добро», которому учат точки над ресурсами
greedy-resources	вариант с наибольшим ростом видимых ресурсов	прагматик
greedy-qualities	вариант с наибольшим ростом скрытых качеств	оракул, верхняя граница, не игрок
human-like	60% costly, 40% greedy- resources, с вероятностью 15% — случайно	целевой игрок, по нему калибруются пороги

Метрики по каждой политике: распределение концовок, день срабатывания каждого озарения, число кризисов за прогон, пустые слоты, снятые отложенные, непоказанные situation, показ трёх притч. У каждой метрики есть цель в «Критериях приёмки».

Сохранения:

- `localStorage`, автосейв после каждого действия, один слот, запрос `navigator.storage.persist()`.
- «Продолжить»: `phase` и `current` восстанавливают экран как был. Отбор чистый, а RNG без состояния, поэтому карточка слота пересчитывается та же и без `current`.
- `version` + миграции схемы.
- Другой `contentVersion` при загрузке: из `scheduled`, `shown`, `history`, `pendingCrises`, `journal` удаляются неизвестные id. Если неизвестна `current.cardId`, слот тянется заново.
- **Импорт** принимает недоверенный JSON, а Zod в рантайме нет. `validateImport: JSON.parse` в try/catch, затем ручная проверка типов и диапазонов (ресурсы 0-100, качества -20...+20, `day` 1-10, `slot` 0-3, `phase` из

списка, массивы строк), потом `sanitize`. Любая ошибка — импорт отклонён, текущее сохранение не трогается.

- **Метрики плейтеста** (`PlaytestMetrics`) лежат в отдельном ключе и не стираются «Начать заново» и сбросом прогресса. Хранится первое решение на концовке и история первого прохождения.
- Доступ в `try/catch`: если хранилище недоступно, игра работает без сохранения и предупреждает об этом.
- **iOS**. В Safari и в установленном PWA хранилища разные, а данные неустановленного сайта Safari может удалить (ИТР). При первом запуске в Safari — подсказка установить игру на экран «Домой» до начала. В настройках — экспорт и импорт прогресса для игрока (файл или текстовый код).

Debug-режим (`?debug=1`): скрытые качества и флаги, выбор любой карточки и дня, изменение значений, сброс, экспорт сохранения и `PlaytestMetrics`.

Ограничения:

- Браузеры: iOS Safari 16+, Chrome Android 110+, актуальные десктопные.
- JS до 150 КБ gzip (React + Framer Motion с `LazyMotion` и `domMax` — около 90–110 КБ), иллюстрации WebP до 60 КБ каждая.
- Сетевой аналитики в MVP нет: метрики выгружаются из debug-режима на плейтесте.
- Язык — русский. Все тексты в контенте и в одном файле строк интерфейса, чтобы потом добавить локализации.

Экраны и UI/UX

Игра — один основной экран с карточкой и несколько полноэкранных переходов. Портретная ориентация, на десктопе — колонка до 480 px по центру.

Экран	Содержимое
Старт	название, «Начать путь» / «Продолжить», «Записи», настройки
Подсказка iOS	при первом запуске в Safari: как установить на экран «Домой» и зачем (иначе прогресс не перенесётся)
Игровой	сверху 4 иконки ресурсов с заливкой; в центре карточка: иллюстрация, имя, текст; внизу глава и день

Экран	Содержимое
Утро	текст дня без цифр или вариант по ведущему качеству (правило — в «Контенте MVP»)
Вечер	итог дня, мудрости дня, затем озарение, если есть: особая анимация (свет, расширение рамки), название, текст
Переход главы	номер и название главы, одна фраза
Концовка	портрет героя текстом; две равнозначные кнопки без выделения по умолчанию — «Продолжить путь» (в MVP — экран «Следующая глава готовится») и «Закончить»; первое нажатие пишется в <code>PlaytestMetrics</code> ; ниже — «Записи» и «Начать заново»
Записи	дневник: собранные мудрости и озарения по дням
Настройки	размер текста, уменьшение анимаций, экспорт и импорт прогресса, сброс с подтверждением в интерфейсе

Взаимодействие с карточкой:

1. Игрок тянет карточку влево или вправо, она наклоняется.
2. При наклоне показывается подпись варианта (какой вариант слева, решает движок) и точки над затронутыми ресурсами.
3. Отпустил дальше порога (35% ширины) — выбор принят, иначе карточка возвращается.
4. Ресурсы плавно меняют заливку, появляется следующая карточка.

Доступность:

- Альтернатива свайпу: две кнопки под карточкой, на десктопе — стрелки ← →.
- Шрифт от 16 px, контраст WCAG AA, зоны нажатия от 44 px.
- Учёт `prefers-reduced-motion`.
- Звука в MVP нет.

Критерии приёмки

Сборка принята, когда выполнены пункты «Сборка» и «Баланс». Цель MVP проверяется отдельно, на плейтесте.

Сборка:

- ☐ Главы 1–2 проходятся от старта до концовки на iPhone (Safari и установленное PWA) и Android (Chrome).
- ☐ Эффекты применяются из данных, ресурсы в пределах 0–100, кризисы — в порядке из раздела «Ресурсы и скрытые качества»; разрешённый кризис не ставится в очередь повторно.
- ☐ Тест: `drawCard(state)` идемпотентна и даёт ту же карточку и стороны после `save/load`.
- ☐ Отложенные карточки выпадают при `day <= сегодня`; карточки с `mustShowBy` показываются не позже своего дня.
- ☐ Озарение показывается на экране вечера и не занимает слот.
- ☐ Скрытые качества не видны вне debug-режима.
- ☐ Закрыть вкладку на любом экране → «Продолжить» открывает тот же экран и ту же карточку с теми же сторонами.
- ☐ Сохранение на прошлой версии контента (удалённая карточка в `scheduled` и `current`) загружается без ошибки.
- ☐ Импорт битого JSON и JSON со значениями вне диапазона отклоняется, текущее сохранение цело; экспорт из Safari → импорт в установленное PWA восстанавливает прогресс.
- ☐ `PlaytestMetrics` переживают «Начать заново» и сброс прогресса.
- ☐ После первого запуска игра работает офлайн и устанавливается на домашний экран.
- ☐ `check-content.ts` проходит все проверки: 0 мёртвых флагов, 0 недостижимых карточек, стоп-лист без ложных срабатываний на тестовом наборе.
- ☐ Движок покрыт юнит-тестами: условия, отбор, стороны, эффекты, кризисы, tie-break ведущего качества, озарения, концовки, `sanitize` и `validateImport`.
- ☐ Debug-режим работает по списку из «Технической части».
- ☐ День проходится за 1,5–2,5 минуты, вся игра — за 15–25 минут.

Баланс (`simulate.ts`, 1000 прогонов на политику):

Метрика	Политики	Цель
пустые слоты	все	0
показаны все 3 притчи	все	100% прогонов

Метрика	Политики	Цель
«Первый шаг»	human-like	30–70% (по ней калибруется порог суммы)
«Первый шаг»	always-costly	не больше 80%: «дорогое = добро» не должно быть выигрышной стратегией
распределение концовок	always-first vs always-second	разница не больше 15 п.п. (нет авторского смещения)
день первого озарения, медиана	greedy-qualities	не раньше 7-го; оба озарения достижимы
кризисы за прогон, в среднем	все, кроме always-costly	не больше 4
снятые отложенные	human-like	не больше 5% запланированных
показ каждой situation (кроме карточек озарений)	human-like	не меньше 50% прогонов

Цель MVP (плейтест, 5–10 человек; на такой выборке это сигнал, а не статистика):

- ☐ Не меньше 60% тестеров называют отложенное последствие своего выбора, и оно подтверждается `history` из экспорта.
- ☐ Не меньше 60% первым нажимают «Продолжить путь», а не «Закончить» (`PlaytestMetrics.endingChoice`), и на вопрос 5 отвечают «да».

Протокол интервью после прохождения, около 10 минут. Вопросы идут от свободных к наводящим, названные последствия сверяются с `history` :

1. Перескажи своими словами, что происходило с героем.
2. Был ли выбор, последствия которого ты увидел позже? Какой?
3. Был ли момент, когда игра, по-твоему, тебя оценила или поучала? Где?
4. Что изменилось в герое или в мире к концу?
5. Сыграл бы ты следующую главу? Что хочешь в ней увидеть?

Этапы и оценка сроков

Код — 12–14 дней. Проект целиком — 3,5–4 недели, если контент и иллюстрации делает не разработчик, и около 5 недель, если всё делает один человек (шаги 4–5 и 6 тогда идут последовательно). Симулятор и политики идут до основного контента: от них зависят все пороги.

1. **Движок и модель данных** — 3 дня: типы, Zod-схемы, условия, чистый отбор с `mustShowBy`, стороны, кризисы, эффекты, RNG, тесты, `check-content.ts`.
2. **Дизайн чернового контента** — 1–2 дня: 53 карточки без текстов, граф флагов и chain, числа для 106 вариантов, пары конфликтов.
3. **`simulate.ts` с политиками** — 2 дня, на черновом контенте. На выходе — пороги, сброс кризиса и диапазоны эффектов, удовлетворяющие таблице «Баланс».
4. **Игровой экран и свайп** — 3 дня: карточка, ресурсы, точки-подсказки, кнопки и клавиатура.
5. **Системные экраны, сохранения с миграциями, экспорт и импорт с валидацией, метрики, PWA с тестом на iOS, debug** — 4–5 дней.
6. **Контент глав 1–2** — 7–8 дней, после шага 3; критический путь. Тексты для 53 карточек (106 вариантов), 16 мудростей (с тестом перестановки), 25 текстов дня, 2 озарения, 2 концовки.
7. **Иллюстрации** — параллельно: герой + 6 персонажей, иконки ресурсов, иконка приложения.
8. **Плейтест и баланс** — 5 дней: 5–10 тестеров, интервью по протоколу, повторный прогон `simulate.ts` на финальном контенте.